



新的文件组件：Path

北京理工大学计算机学院
金旭亮

Path实例的创建-1



Paths类提供了一个get方法，可以用于创建Path实例，它能正确地处理不同操作系统路径分隔符的问题：

```
Path path1 = Paths.get("c:\\windows");  
Path path2 = Paths.get("c:/windows");  
Path path3 = Paths.get("c:", "windows");
```



推荐在项目中使用 “/” 取代 “\\” 。



类似于，Path类提供了一个of方法，也能方便地创建Path实例：

```
Path path1 = Path.of("c:\\windows");  
Path path2 = Path.of("c:/windows");  
Path path3 = Path.of("c:", "windows");
```

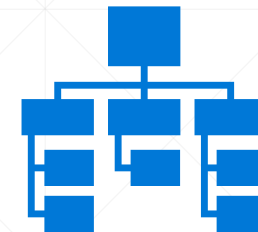
Path实例的创建-2



Path实例还可以通过FileSystem对象来创建。

```
FileSystem fileSystem = FileSystems.getDefault();  
// "c:/windows" 和 "c:\\windows" 返回相同的Path对象  
Path examplePath = fileSystem.getPath("c:/windows");
```

文件系统 (FileSystem)



文件系统是操作系统级别的概念，它将数据保存于文件与文件夹中，并且负责管理它们。



文件系统有一个“根 (root)”文件夹的概念，在Linux中，每个mount的设备都是一个文件系统，在Windows中，驱动器 (C:、D:) 等构成了一个文件系统。

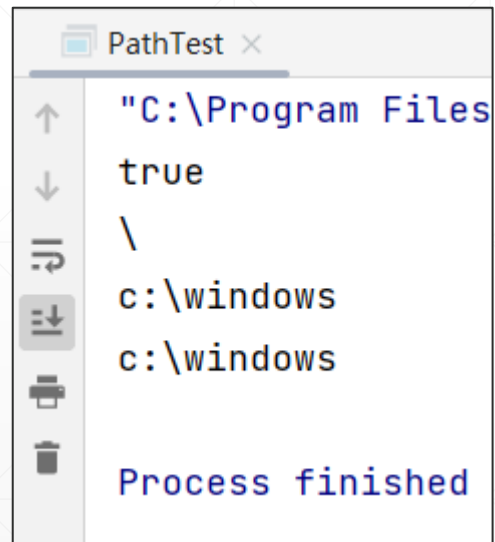


文件系统的路径有一个分割符，在Linux中是“/”，Windows中是“\”。

构建Path类实例示例代码

```
//使用FileStstem创建Path实例
private static void createPathUseFileSystem() {
    FileSystem fileSystem = FileSystems.getDefault();
    Path windowsPath = fileSystem.getPath( first: "c:/windows");
    Path windowsPath2 = fileSystem.getPath( first: "c:\\windows");
    //上述两个路径, 其实是一样的
    System.out.println(windowsPath.equals(windowsPath2));
    //获取路径分隔符, Windows下是"\"
    System.out.println(fileSystem.getSeparator());
    //不管是使用"\"还是"/"构建Path实例, 最后输出的, 都是当前操作系统默认的分隔符
    System.out.println(windowsPath.toString()); //c:\windows
    System.out.println(windowsPath2.toString()); //c:\windows
}
```

PathTest.java



File --> Path



传统File, 转变为Path:

```
private static void fileObjToPathObj() {  
    File windowsDir = new File(pathname: "c:\\windows");  
    //可以把File转换为Path  
    Path windowsPath = windowsDir.toPath();  
    System.out.println(windowsPath);  
}
```



```
Run: PathTest x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
c:\windows  
  
Process finished with exit code 0
```

获取当前文件夹

```
1 usage
private static void getCurrentDir() {
    System.out.println(System.getProperty("user.dir"));
    System.out.println(Path.of("").toAbsolutePath());
    System.out.println(Paths.get(".").normalize().toAbsolutePath());
}
```

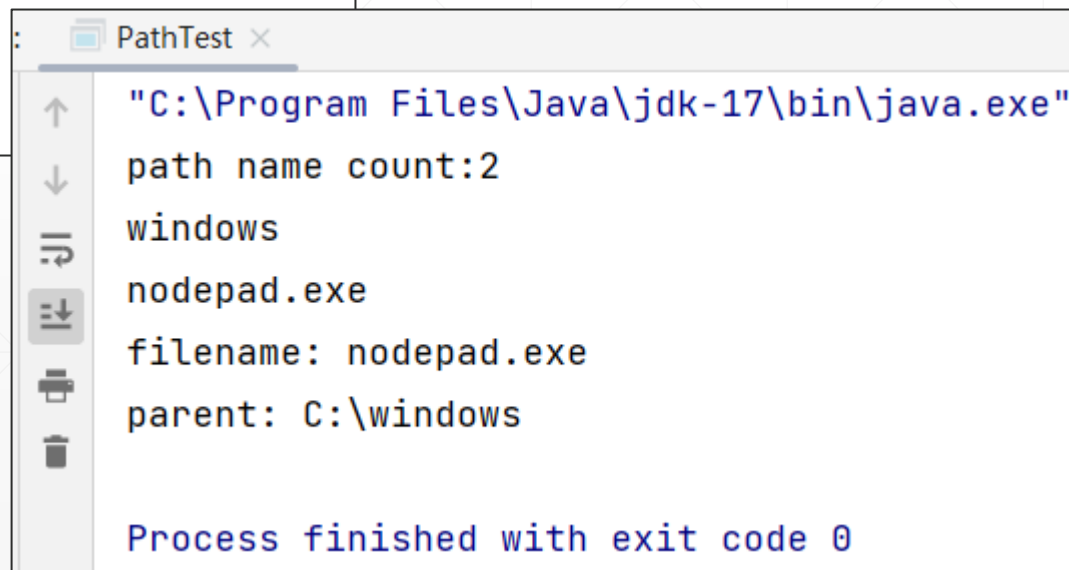
得到当前文件夹的完整路径之后，就可以使用它来“拼接”出特定文件的物理路径。
比如，一张图片位于项目的images文件夹下，就可以使用以下代码得到它的“物理路径”：

当前文件夹 + `"/images/myimage.jpg"`

使用Path解析路径字符串

```
private static void getPathSegmentInfo() {  
    Path path = Paths.get("C:/windows/nodepad.exe");  
    System.out.println("path name count:" + path.getNameCount()); //2  
    System.out.println(path.getName(0)); //windows  
    System.out.println(path.getName(1)); //nodepad.exe  
    //filename: nodepad.exe  
    System.out.println("filename: " + path.getFileName());  
    //parent: C:\windows  
    System.out.println("parent: " + path.getParent());  
}
```

Path可以将“文件路径”字符串按照文件分隔符“切成”一段一段地，分开处理，这在实际开发中很方便.....



```
PathTest x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
path name count:2  
windows  
nodepad.exe  
filename: nodepad.exe  
parent: C:\windows  
  
Process finished with exit code 0
```


“.”和“..”，绝对路径与真实路径

```
private static void realPathAndAbsolutePath() throws IOException {  
    //“.”表示当前文件夹  
    Path currentDir = Paths.get( first: ".");  
    //样例输出: C:\Java NIO\PathAndFiles\  
    System.out.println(currentDir.toAbsolutePath());  
    //样例输出: C:\Java NIO\PathAndFiles\  
    System.out.println(currentDir.toRealPath());  
    //“..”表示当前文件夹的上一级文件夹  
    Path currentParent = Paths.get( first: "..");  
    //样例输出: C:\Java NIO  
    System.out.println(currentParent.toRealPath());  
  
    //如果文件夹不存在  
    Path notExistPath = Paths.get( first: "c:\\notExist");  
    //C:\notExist  
    System.out.println(notExistPath.toAbsolutePath());  
    //抛出NoSuchFileException  
    System.out.println(notExistPath.toRealPath());  
}
```



toAbsolutePath()方法，将得到一个绝对路径，但这个路径是否对应“真实存在”的文件或文件夹，这个是不知道的。

toRealPath()方法会先获取绝对路径，然后检查这个路径所对应的文件或文件夹是否“真实存在”，不存在的话，会抛出一个异常。。

路径的标准化

这个实际上代表了一个“路径”的移动过程.....

```
private static void normalize() {  
    Path somePath = Paths.get( first: "c:/windows/./System32/drivers/../../");  
    //normalize会移除“.”和“..”  
    //输出 c:\windows\System32  
    System.out.println(somePath.normalize());  
}
```



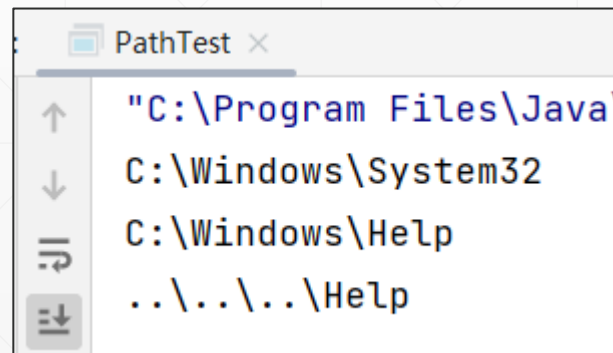
```
Run: PathTest x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
c:\windows\System32  
Process finished with exit code 0
```

`normalize()`方法干的活，其实就是移除路径字符串中那些不必要的部分（比如“.”和“..”），得到一个规范的路径字符串，但它并不检查这个路径字符串是否“真的存在”。

相对路径解析

```
private static void resolveAndRelativize() throws IOException {  
    Path windowsPath = Paths.get( first: "C:", ...more: "windows");  
    //解析子文件夹  
    Path system32Path = windowsPath.resolve("System32");  
    //C:\windows\System32  
    System.out.println(system32Path.toRealPath());  
    //解析兄弟文件夹  
    Path siblingPath = system32Path.resolveSibling("Help");  
    //C:\Windows\Help  
    System.out.println(siblingPath.toRealPath());  
    //生成从一个Path到另一个Path的“转移路线”  
    Path etcPath = Paths.get( first: "C:\\Windows\\System32\\drivers\\etc");  
    Path helpPath = Paths.get( first: "C:\\Windows\\Help");  
    Path relativzieResult = etcPath.relativize(helpPath);  
    //输出: ..\\..\\..\\Help  
    System.out.println(relativzieResult);  
}
```

如果resolve接收的是一个绝对路径，则直接返回这个绝对路径。



区分Path和URI

```
private static void pathAndUri() throws IOException {  
    Path path = Paths.get( first: ".");  
    System.out.println(path.toUri());  
    System.out.println(path.toRealPath());  
}
```



```
Run: Main x  
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"  
file:///D:/JavaProjects/ExploreNIOFileSystem/./  
D:\JavaProjects\ExploreNIOFileSystem  
Process finished with exit code 0
```

URI称为“统一资源标识符”，是一种业界通用的规范，可以用于标识本地和互联网上的资源。

Path则专用于本地文件系统。

多了解一点：关于IntelliJ中的文件路径



使用IntelliJ开发Java项目，放在src文件夹中的.java文件会被编译，而其他类型的文件，则会被复制到输出文件夹。



资源文件夹中的文件，也会被复制到输出文件夹。



所以，可以在代码中通过名字直接访问到这些文件。

1 usage

```
private static void useResourceFile() {  
    var pathDemo = new PathTest();  
    var dataFile = pathDemo.getClass().getResource(name: "data.txt");  
    System.out.println(dataFile);  
    var testFile = pathDemo.getClass().getResource(name: "test.txt");  
    System.out.println(testFile);  
}
```

